QATIPv3 AWS Lab4
Variables and Workspaces

Contents

Lab Objectives 1
Teaching Points 1
Before you begin 2
Solution 2
Task 1 - Review provisioned terraform files 2
Task 2 - Run terraform plan and apply with hardcoded parameter values 3
Task 3 - Substitute hardcoded parameter values with variables 4
Task 4 - Overriding variable values at the command prompt 5
Task 5 - Override variable values using terraform.tfvars 6
Task 6 - Implement Terraform Workspaces 7
Task 7 - Lab Clean-up 10
Terraform Workspaces in Real-World Deployments 12

Lab Objectives

In this lab you will:

- Deploy resources using a hard-coded terraform file
- Utilize variables to make your code reuseable
- Implement overrides using the command line and tfvars files
- Utilize terraform workspaces to allow multiple simultaneous deployments

Teaching Points

When writing terraform code, it is instinctive to supply explicit values when they are required, for example **name = "michael"**. This can make the code very static though, requiring manual changes of these values each time a modification is

needed. A better practice is to use 'placeholders', variables, to which values can be passed at runtime, for example **name = var.username**. Passing values to these variables can be achieved using variable file default values, tfvars files, command line supplied or by being prompted. By default, all changes made to your configuration will be tracked by a single state file. Workspaces allow multiple state files to exist simultaneously so that the same base code, provided with unique variable values, can be deployed. This makes your code flexible and reuseable.

Before you begin

1. Ensure you have completed Lab0 before attempting this lab.
2. In the IDE terminal pane, enter the following commands…

   cd c:\awslabs\04

3. This shifts your current working directory to awslabs/labs/04. Ensure all commands are executed in this directory
4. Close any open files and use the Explorer pane to navigate to and open the pre-configured main.tf file in awslabs/04.

Solution

There is no solution code for this lab as it involves multiple deployment phases. Reach out to your instructor if you encounter issues.

Task 1- Review provisioned terraform files

1. Review the provisioned files in awslabs/04..

   main.tf Hard coded deployment of an t2.micro EC2 instances in "**us-west-2**". This represents the type of resource we will create in this lab, in production there would be many other resources defined here. There is also an output block (lines 28-37) which displays information about the deployment, including the workspace.

   variables.tf All content except 'region' currently commented out

   terraform.tfvars All content currently commented out

dev.tfvars Variable values that will be used in a new workspace

prod.tfvars Variable values that will be used in a new workspace

Task 2- Run terraform plan and apply with hardcoded parameter values

1. Ensure you have navigated to the **awslabs/04** folder
2. Run **terraform init**
3. Run **terraform plan**
4. Review the plan output..

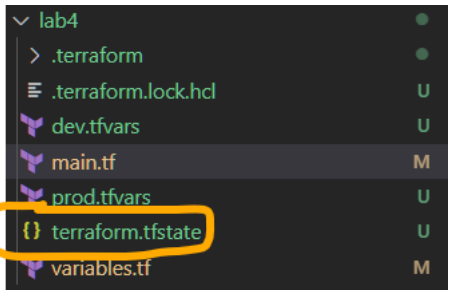
```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_details = {
      + ami       = "ami-0d08c3b92d0f4250a"
      + location  = "us-west-2"
      + size      = "t2.micro"
      + vm_names  = [
          + "terraform-demo-default-01",
        ]
      + workspace = "default"
    }
```

5. Run **terraform apply** followed by **yes**
6. Switch to the Console and verify the creation of the EC2 instance in us-west-2 (your availability-zone may differ)..



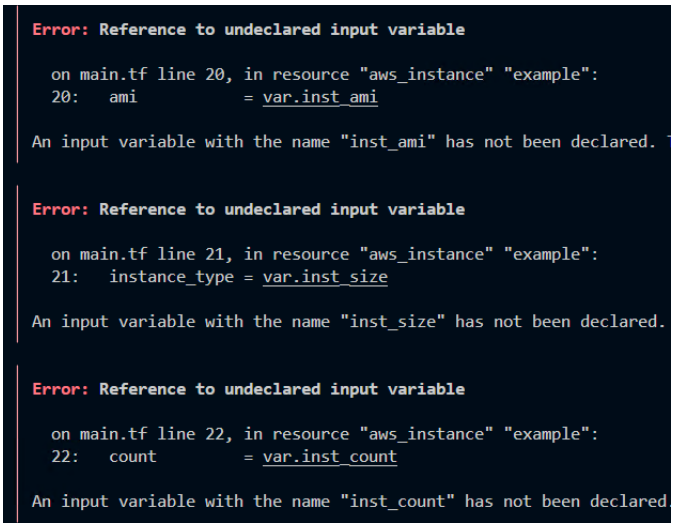
7. Note the creation of a state file in the root folder..



8. **Takeaway**: Hardcoded parameter values will always be used if they exist. This can make the code inflexible and static

Task 3- Substitute hardcoded parameter values with variables

1. In **main.tf**; Replace hard-coded region "us-west-2" with **var.region** (no quotes)
2. Replace hard-coded ami "ami-0d08c3b92d0f4250a" with **var.inst_ami**
3. Replace hard-coded instance_type “t2.micro” with **var.inst_size**
4. Replace hard-coded count 1 with **var.inst_count**
5. Save the changes.
6. Run **terraform plan** and review the errors…



7. **Takeaway**: If variables are referenced in your code, then they **must** be declared.
8. In **variables.tf**; uncomment lines 7 through 23 **except** line 10 which provides default values for the inst_count variable.
9. Save the changes.
10. Run **terraform plan**
11. When prompted, enter **1** as the number of instances.
12. Planning and Apply operations complete using the variable default values if they exist. prompted for when there is no default value. Given that these

values are the same as the old static values, the planning phase shows that there are no changes needed

13. In **variables.tf**; uncomment line 10 and save the file

14. Run **terraform plan**

15. The planning completes using all variable values drawn from the variables file. Given that these values are the same as the old static values, the planning phase shows that there are no changes needed.

16. Destroy your deployment using **terraform destroy** followed by **yes**

17. Switch to the console and confirm the deletion of the default instance in us-west-2

18. **Takeaway**: If a variable is declared but no value has been assigned then you are prompted for values. If a variable is declared with a default value then this value will be used unless overridden.

Task 4- Overriding variable values at the command prompt

1. Enter the following command..

   terraform plan -var="inst_size=t3.micro" -var="inst_count=2"

```
Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_details = {
      + ami       = "ami-0d08c3b92d0f4250a"
      + region    = "us-west-2"
      + size      = "t3.micro"
      + vm_names  = [
          + "terraform-demo-default-01",
          + "terraform-demo-default-02",
        ]
      + workspace = "default"
    }
```

   Takeaway: Supplying variable values at the command prompt using **-var=" "** has the highest priority and overrides values supplied ***anywhere*** else. In this case **t3.micro** is used as the instance type value and **2** instances would be

created. The ami and region values are drawn from the variables file default values. Do not apply this deployment.

Task 5- Override variable values using terraform.tfvars

1. Uncomment lines 3 to 5 in **terraform.tfvars**. This supplies values for region, size and ami which will take precedence over any default values in variables.tf

```
1  #update with appropriate amis when needed
2  #inst_count = 2
3  region = "us-west-2"
4  inst_size = "t3.micro"
5  inst_ami = "ami-0d08c3b92d0f4250a"
```

   Save the changes

2. Run **terraform plan**

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_details = {
      + ami       = "ami-0d08c3b92d0f4250a"
      + region    = "us-west-2"
      + size      = "t3.micro"
      + vm_names  = [
          + "terraform-demo-default-01",
        ]
      + workspace = "default"
    }
```

   Notice that the region, size and ami values are now drawn from terraform.tfvars, overriding the default values in variables.tf if they conflict. Here there is a size conflict between t2.micro in variables.tf and t3.micro in terraform.tfvars. Note that only one vm would be created as the count value is still drawn from variable.tf

3. In variables.tf, remove defaults from all variables and save the changes.

```
variable "region" {
  description = "AWS region"
  type        = string
}

variable "inst_count" {
  description = "Number of instances"
  type        = number
}

variable "inst_size" {
  description = "Instance size"
  type        = string
}

variable "inst_ami" {
  description = "Instance AMI"
  type        = string
}
```

4. Run **terraform plan**

   There is no value defined for 'count' in variables.tf or terraform.tfvars and therefore you are prompted for a value

5. Enter '**2**'

```
Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_details = {
      + ami       = "ami-0d08c3b92d0f4250a"
      + location  = "us-west-2"
      + size      = "t3.micro"
      + vm_names  = [
          + "terraform-demo-default-01",
          + "terraform-demo-default-02",
        ]
      + workspace = "default"
    }
```

   Planning proceeds with values drawn from terraform.tfvars and prompts

6. In terraform.tfvars, uncomment line 2 and verify that planning completes without prompting, all values being derived from terraform.tfvars.

7. Run **terraform apply** followed by **yes** and then switch to the portal to verify the deployment of 2 t3.micro instances in us-west-2.

   Takeaway: Terraform.tfvars, if it exists, is referenced automatically during plan and apply operations. Its values override any default values that may be defined when the variable is declared. Therefore, if values are contained in terraform.tfvars, there is no need to decalre a default value as it will always be overridden.

Task 6- Implement Terraform Workspaces

1. Changing deployment parameters will affect the current deployment. Workspaces allow multiple deployments, using the same base code but with different parameters, to exist simultaneously, each with its own state file. If no new workspaces are created then your deployment is in the **default** workspace which always exists and cannot be deleted.
2. Create two workspaces; "**development**" and "**production**"…
   terraform workspace new development

terraform workspace new production
terraform workspace list

```
awsstudent:~/environment/awslabs/04 (main) $ terraform workspace list
  default
  development
* production
```

3. The * indicates your current workspace is now **production**. The **default** workspace always exists, and this is where your current deployment of 2 t3.micro instances in us-west-2 is tracked by the state file **terraform.tfstate** in the root folder.

4. Note the creation of a new folder "**terraform.tfstate.d**" with an empty subfolder for each of the new workspaces..



5. Review **prod.tfvars**

```
> 04 > prod.tfvars > ...
#update with appropriate amis when needed
region      = "us-east-2"
inst_count  = 3
inst_size   = "t3.small"
inst_ami    = "ami-00a9f44477dd83e3d"
```

All four variable values are defined here but are not in use until explicitly invoked.

6. Run **terraform plan --var-file=prod.tfvars**

```
Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_details = {
      + ami       = "ami-00a9f44477dd83e3d"
      + location  = "us-east-2"
      + size      = "t3.small"
      + vm_names  = [
          + "terraform-demo-production-01",
          + "terraform-demo-production-02",
          + "terraform-demo-production-03",
        ]
      + workspace = "production"
    }
```

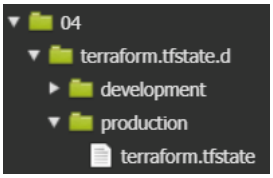
If there are conflicts, values from **prod.tfvars** override default values and values specified in **terraform.tfvars**.

Notice that the plan will not propose to destroy any resources. Recall that we currently have 2 t3.micro instances deployed in us-west-2. This is in the default workspace and will not be affected as we are now in the production workspace.

7. Run **terraform apply --var-file=prod.tfvars** followed by **yes**

8. Switch to the console and verify the creation of the 3 t3.small instances in us-east-2. Switch to us-west-2 to confirm the 2 t3.micro instances created in the default workspace still exists

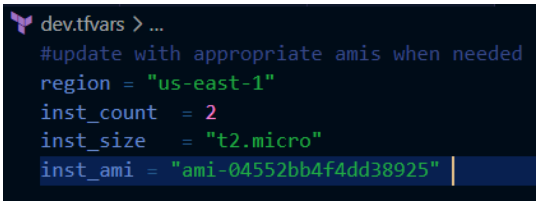
9. In the IDE, expand the terraform.tfstate.d folder and verify the creation of a new state file for the production workspace..



10. Move to the development workspace..

    terraform workspace select development

11. Review **dev.tfvars**



    Again, all four variable values are defined but are not in use until explicitly invoked.

12. Run **terraform plan --var-file=dev.tfvars**

```
Plan: 2 to add, 0 to change, 0 to destroy.

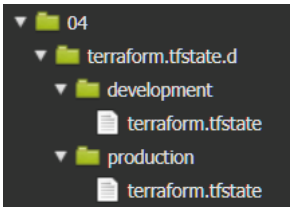
Changes to Outputs:
  + instance_details = {
      + ami       = "ami-04552bb4f4dd38925"
      + location  = "us-east-1"
      + size      = "t2.micro"
      + vm_names  = [
          + "terraform-demo-development-01",
          + "terraform-demo-development-02",
        ]
      + workspace = "development"
    }
```

Notice again that the plan will not destroy any existing resources as we are now in a new.

13. Run **terraform apply --var-file=dev.tfvars** followed by **yes**

14. Switch to the console and verify the creation of the 2 t2.micro instances in us-east-1 alongside the existing default and production workspace deployments in us-west-2 and us-east-2.

15. In the IDE, expand the terraform.tfstate.d folder and verify the creation of a new state file for the development workspace..



Task 7- Lab Clean-up

1. When deleting resources in workspaces, always pay close attention to the workspace you are currently working in. Use **terraform workspace show** to determine your current workspace..

```
awsstudent:~/environment/awslabs/04 (main) $ terraform workspace show
development
awsstudent:~/environment/awslabs/04 (main) $
```

2. Destroy the resources in the current workspace (dev) using..
   terraform destroy

3. Enter **yes** when prompted

4. Switch to the Console to confirm the deletion of the 2 development instances in us-east-1.

5. The current workspace cannot be deleted so move to the **production** workspace and delete the **development** workspace…

   terraform workspace select production
   terraform workspace delete development

6. Destroy the resources in the current workspace (production) using..
   terraform destroy --var-file=prod.tfvars

7. Enter **yes** when prompted

8. Switch to the console to confirm the deletion of the 3 production instances in us-east-2

9. The current workspace cannot be deleted. Move to the **default** workspace and delete the **production** workspace…

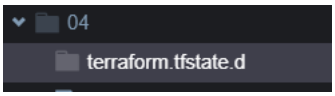
   terraform workspace select default
   terraform workspace delete production

10. Verify the deletion of the workspaces, leaving just the default workspace. This cannot be deleted..

    terraform workspace list

```
awsstudent:~/environment/awslabs/04 (main) $ terraform workspace list
* default
```

11. Note that the workspace directories are deleted along with the workspaces themselves…



12. Destroy the resources in the current workspace (default) using **terraform destroy** followed by **yes**

13. Enter **yes** when prompted

14. Use the console to confirm the deletion of the 2 instances in us-west-2

Terraform Workspaces in Real-World Deployments

In this lab, Terraform workspaces were used to demonstrate how the same Terraform codebase can maintain multiple independent deployments simultaneously. Each workspace maintains its own separate state file, allowing different infrastructure deployments to coexist without interfering with one another.

For learning and smaller environments, workspaces provide a lightweight and convenient mechanism for separating deployments such as development, test and production. Combined with tfvars files, this approach allows the same Terraform configuration to be reused with different deployment settings including regions, instance sizes and instance counts.

It is important to understand however that Terraform workspaces only isolate Terraform state. They do not inherently isolate permissions, governance processes or deployment workflows.

As Terraform deployments grow in size and importance, organisations often introduce additional operational controls around how infrastructure changes are deployed. These topics are explored later in the course when CI/CD pipelines and automated deployment workflows are introduced.

At this stage, workspaces should be viewed as a useful mechanism for managing multiple deployments from the same Terraform codebase while reinforcing the relationship between configuration, variables and Terraform state.

Congratulations, you have completed this lab